Due Date: April 28th, 23:59, 2026

Instructions

- *This assignment is computationally involved. Start early and budget time for training and debugging.*
- *Submit your report (PDF) and the required code files electronically via the course Gradescope page.*
- *Implementation details and starter code are provided in the accompanying notebooks. The PDF release only summarizes the required tasks.*
- *For report questions, concise analysis and clearly presented figures are more important than long answers.*
- *You should use Google Colab if you do not have convenient GPU access locally.*
- *You are encouraged to ask questions on the Piazza. TA for this assignment* ***Jingyue Zhang****.*

Summary: This assignment has three programming problems. In Problem 1, you will implement a Denoising Diffusion Probabilistic Model (DDPM) on FashionMNIST. In Problem 2, you will implement a Flow Matching model and compare ODE-based generation with DDPM. In Problem 3, you will work on language-model alignment with reward modeling, Direct Preference Optimization (DPO), and Best-of-N sampling.

Coding instructions You will use PyTorch throughout the assignment. Follow the notebooks closely for implementation guidance and experimental setup.

Submission Submit the required code files for each problem to Gradescope, together with a PDF report covering all report questions.

Image Generation

Problem 1

Denoising Diffusion Probabilistic Models (35 pts) In this problem, you will implement a Denoising Diffusion Probabilistic Model (DDPM)[1] to generate **FashionMNIST** images. The U-Net[2] backbone is already provided. Your work is to complete the diffusion-model utilities in `q1_ddpm.py` and the sampling / visualization code in `q1_trainer_ddpm.py`. Follow the guided workflow in `01 - DDPM.ipynb`; Gradescope will use the Python files rather than the notebook itself.

The shipped code intentionally keeps the W25 DDPM contract, including the same linear beta schedule and method names, so the grader-facing behavior remains stable.

Dataset reference: https://github.com/zalandoresearch/fashion-mnist

You will use PyTorch throughout. Useful references include `torch.cumprod`, `torch.randn`, and `torch.randint`.

Background Given a clean image $\mathbf{x}_0 \sim q(\mathbf{x})$, the forward diffusion process adds Gaussian noise over T steps:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\Big(\mathbf{x}_t; \sqrt{1-\beta_t}\,\mathbf{x}_{t-1}, \beta_t \mathbf{I}\Big).$$

Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$. Then the closed-form forward marginal is

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I}\big),$$

which yields the reparameterization

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

The reverse model predicts the noise with a U-Net $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ and defines

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}\big),$$

with

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right), \qquad \sigma_t^2 = \beta_t.$$

The simplified DDPM training objective is

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}}\left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2\right],$$

where t is sampled uniformly from $\{0, \ldots, T-1\}$.

[1] Ho et al., *Denoising Diffusion Probabilistic Models*, NeurIPS 2020. https://arxiv.org/abs/2006.11239

[2] Ronneberger et al., *U-Net: Convolutional Networks for Biomedical Image Segmentation*, MICCAI 2015. https://arxiv.org/abs/1505.04597

Configuration

- Dataset: FashionMNIST, resized to $1 \times 32 \times 32$
- Number of diffusion steps: $T = 1000$
- Beta schedule: linear
- Batch size: 256
- Learning rate: 2×10^{-4}
- Epochs: 20
- Optimizer: Adam

1. (3 pts) Implement `q_xt_x0(self, x0, t)` in `q1_ddpm.py` to return the mean and variance of $q(\mathbf{x}_t \mid \mathbf{x}_0)$.
2. (3 pts) Implement `q_sample(self, x0, t, eps)` using the reparameterization formula above.
3. (4 pts) Implement `p_xt_prev_xt(self, xt, t)` to return the mean and variance of $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$.
4. (3 pts) Implement `p_sample(self, xt, t)` to sample $\mathbf{x}_{t-1}$ from the reverse model.
5. (5 pts) Implement `loss(self, x0, noise)` for the simplified DDPM objective.
6. (5 pts) Complete `Trainer.sample` in `q1_trainer_ddpm.py` to run the full reverse diffusion loop from pure noise down to a generated sample.
7. (2 pts) Complete `Trainer.generate_intermediate_samples` to save selected reverse-process states for visualization.
8. (5 pts, report) Train the model and show generated sample grids from several epochs. Comment on sample sharpness, diversity, common failure modes, and practical changes that would likely improve results.
9. (5 pts, report) Visualize several intermediate denoising stages from the same reverse trajectory and explain how semantic structure emerges from noise over time. Relate your observations to the reverse-process equations above.

Files to submit `q1_ddpm.py`, `q1_trainer_ddpm.py`, and your PDF report.

Problem 2

Flow Matching (27 pts) In this problem, you will implement a Flow Matching[3] model on the same FashionMNIST using the same U-Net backbone as in Problem 1. Instead of reversing a discrete stochastic diffusion chain, Flow Matching learns a continuous velocity field and generates samples by integrating an ODE backward from noise to data. Follow `02 - Flow Matching.ipynb`; Gradescope will read `q2_flow_matching.py` and `q2_trainer_fm.py`.

Background Flow Matching learns a velocity field $\mathbf{v}_\theta$ satisfying

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}_\theta(\mathbf{x}_t, t), \qquad t \in [0, 1].$$

For a clean image $\mathbf{x}_0$ and a Gaussian noise sample $\mathbf{x}_1 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, use the straight-line interpolation

$$\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1,$$

with conditional target velocity

$$\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0, \mathbf{x}_1) = \mathbf{x}_1 - \mathbf{x}_0.$$

The Flow Matching objective is

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t,\mathbf{x}_0,\mathbf{x}_1}\left[\|\mathbf{v}_\theta(\mathbf{x}_t, t) - \mathbf{u}_t\|^2\right], \qquad t \sim \text{Uniform}(0, 1).$$

At sampling time, start from noise $\mathbf{x}_1$ and integrate backward from $t = 1$ to $t = 0$. For a time step $\Delta t < 0$,

$$\mathbf{x}_{t+\Delta t}^{\text{Euler}} = \mathbf{x}_t + \Delta t\, \mathbf{v}_\theta(\mathbf{x}_t, t),$$

and the Midpoint method uses two network evaluations:

$$\mathbf{x}_{t+\Delta t/2} = \mathbf{x}_t + \frac{\Delta t}{2}\mathbf{v}_\theta(\mathbf{x}_t, t), \qquad \mathbf{x}_{t+\Delta t}^{\text{Midpoint}} = \mathbf{x}_t + \Delta t\, \mathbf{v}_\theta(\mathbf{x}_{t+\Delta t/2}, t + \Delta t/2).$$

Configuration

- Dataset: FashionMNIST, resized to $1 \times 32 \times 32$
- Batch size: 256
- Learning rate: 2×10^{-4}
- Epochs: 20
- Optimizer: Adam
- Backbone: the same U-Net as Problem 1

[3] Lipman et al., *Flow Matching for Generative Modeling.* https://arxiv.org/abs/2210.02747

1. (2 pts) Implement `sample_xt` in `q2_flow_matching.py` for the interpolation path $\mathbf{x}_t = (1-t)\mathbf{x}_0 + t\mathbf{x}_1$.
2. (2 pts) Implement `compute_conditional_velocity` for the target velocity $\mathbf{u}_t = \mathbf{x}_1 - \mathbf{x}_0$.
3. (5 pts) Implement `loss`. Sample $t \sim \text{Uniform}(0,1)$, form $\mathbf{x}_t$, compute the target velocity, predict the velocity with the U-Net, and return the MSE summed over non-batch dimensions and averaged over the batch.
4. (3 pts) Implement one Euler integration step in `euler_step`.
5. (5 pts) Implement one Midpoint integration step in `midpoint_step`.
6. (5 pts, report) Compare DDPM and Flow Matching theoretically. Discuss the difference between the DDPM denoising objective and the Flow Matching velocity objective, the difference between stochastic reverse diffusion and deterministic ODE integration, and how the same U-Net is used differently in the two problems.
7. (5 pts, report) Evaluate sampling quality and efficiency. Show Euler samples for multiple step counts, Midpoint samples for multiple step counts, and compare them to your full DDPM baseline using the same starting noise. Report the step counts, network-evaluation budgets, and observed runtime tradeoffs.

Files to submit `q2_flow_matching.py` and your PDF report.

Language Model Alignment

Problem 3

Reward Modeling, DPO, and Best-of-N Sampling (38 pts) In this problem, you will align a language model with human preference data. You will implement a Bradley-Terry[4] reward model, Direct Preference Optimization (DPO)[5], and Best-of-N sampling using **GPT-2 Medium**[6] on a subset of an **instruction-preference** dataset. **PLEASE DOWNLOAD THE ARTIFACT:** artifact download link. Follow `03 - RLHF.ipynb`; Gradescope will read `q3_reward_model.py`, `q3_dpo.py`, and `q3_bon.py`.

[4] Bradley & Terry, *Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons*

[5] Rafailov et al., *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*, https://arxiv.org/abs/2305.18290

[6] Radford et al., *Language Models are Unsupervised Multitask Learners*, 2019. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

Background Each preference example contains a prompt $\mathbf{x}$, a chosen response $\mathbf{y}_w$, and a rejected response $\mathbf{y}_l$. A Bradley-Terry reward model assigns a scalar score $r_\phi(\mathbf{x}, \mathbf{y})$ and models pairwise preference[7] as

$$P(\mathbf{y}_w \succ \mathbf{y}_l \mid \mathbf{x}) = \sigma(r_\phi(\mathbf{x}, \mathbf{y}_w) - r_\phi(\mathbf{x}, \mathbf{y}_l)) .$$

The corresponding reward-model loss is

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(\mathbf{x},\mathbf{y}_w,\mathbf{y}_l)} \left[\log \sigma(r_\phi(\mathbf{x}, \mathbf{y}_w) - r_\phi(\mathbf{x}, \mathbf{y}_l))\right] .$$

The reward model uses GPT-2 as a backbone and maps the last non-padding hidden state to a scalar reward:

$$r_\phi(\mathbf{x}, \mathbf{y}) = \mathbf{w}^\top \mathbf{h}_{\text{last}} + b.$$

For an autoregressive language model π_θ, the response log-probability is

$$\log \pi_\theta(\mathbf{y} \mid \mathbf{x}) = \sum_{t=1}^{T} \log p_\theta(y_t \mid \mathbf{x}, y_1, \ldots, y_{t-1}).$$

In implementation, remember that logits at position t predict the token at position $t+1$, so you must shift logits and labels. Only response tokens, not prompt tokens, should contribute to the sum.

DPO replaces explicit RL policy optimization with a preference loss against a frozen reference model π_{ref}:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(\mathbf{x},\mathbf{y}_w,\mathbf{y}_l)} \left[\log \sigma\left(\beta \left(\log \frac{\pi_\theta(\mathbf{y}_w \mid \mathbf{x})}{\pi_{\text{ref}}(\mathbf{y}_w \mid \mathbf{x})} - \log \frac{\pi_\theta(\mathbf{y}_l \mid \mathbf{x})}{\pi_{\text{ref}}(\mathbf{y}_l \mid \mathbf{x})}\right)\right)\right] .$$

The corresponding implicit reward, up to the prompt-dependent normalization constant, is

$$\hat{r}(\mathbf{x}, \mathbf{y}) = \beta \left(\log \pi_\theta(\mathbf{y} \mid \mathbf{x}) - \log \pi_{\text{ref}}(\mathbf{y} \mid \mathbf{x})\right) .$$

Best-of-N sampling generates N candidate responses, scores each with the reward model, and returns the highest-scoring completion.

Configuration

- Base model: GPT-2 Medium (355M parameters), loaded via `AutoModel.from_pretrained`
- Dataset: Instruction-Preference subset
- Max sequence length: 512

[7] For background on RLHF, see Ouyang et al., *Training language models to follow instructions with human feedback*, NeurIPS 2022. https://arxiv.org/abs/2203.02155

- Batch size: 4, with gradient accumulation of 8
- Optimizer: AdamW
- Reward model hyperparameters: learning rate 1×10^{-5}, 1 epoch
- DPO hyperparameters: learning rate 5×10^{-6}, 1 epoch, $\beta = 0.1$
- A SFT checkpoint is provided and serves as the starting policy and frozen reference model

Part A: Reward Model (12 pts) Implement the reward-model stage of the alignment pipeline.

1. (3 pts) Implement `RewardModel.__init__` and `RewardModel.forward` in `q3_reward_model.py`. Load GPT-2 with `AutoModel.from_pretrained`. Extract the hidden state of the last non-padding token in each sequence and map it to a scalar reward with a `nn.Linear` head.
2. (3 pts) Implement `compute_preference_loss(rewards_chosen, rewards_rejected)` for the Bradley-Terry loss above. Hint: use `F.logsigmoid`.
3. (2 pts) Implement `compute_reward_accuracy` as the fraction of examples where the chosen reward is higher than the rejected reward.
4. (4 pts) Implement `RewardModelTrainer.train_step`. Run the reward model on chosen and rejected sequences, compute the preference loss and accuracy, and return a dictionary with keys `"loss"` and `"accuracy"`.

Part B: DPO Training (15 pts) Implement DPO using a trainable policy model and a frozen reference model.

5. (5 pts) Implement `compute_log_probs` in `q3_dpo.py`. Shift logits and labels, apply `log_softmax`, and gather the log-probabilities of the target next tokens. Mask out prompt tokens and padding, then sum only over response tokens.
6. (5 pts) Implement `compute_dpo_loss`. Compute the chosen and rejected log-ratio advantages relative to the reference model, the DPO loss, the implicit reward margin, and the preference accuracy.
7. (5 pts) Implement `DPOTrainer.compute_loss`. Compute chosen and rejected log-probabilities under both the policy and reference models, use `torch.no_grad()` for the reference model, and return the full DPO loss together with logging metrics.

Part C: Best-of-N Sampling (3 pts) Implement the inference-time ranking stage.

8. (3 pts) Implement `best_of_n_sample` in `q3_bon.py`. Tokenize the prompt, generate N independent completions with `do_sample=True`, score them with the reward model, and decode the best response.

Part D: Report Questions (8 pts) Answer the following questions in your PDF report.

9. (5 pts, report) **DPO vs PPO: theoretical comparison.** Explain:
   - how DPO can optimize directly from preference pairs without training against an explicit reward model during policy optimization;
   - the role of the frozen reference model π_{ref}, and what would happen if the KL regularization strength were removed or $\beta \to 0$;
   - one advantage and one disadvantage of DPO relative to PPO-based RLHF.

   Support your explanation with the DPO objective above rather than only giving an intuitive summary.

10. (3 pts, report) **Best-of-N analysis.**
    - Use at least 5 prompts and report results for $N \in \{1, 4, 8, 16, 32\}$. Plot the mean reward as a function of N and describe the trend.
    - For one prompt, show the Best-of-1 and Best-of-16 responses side by side. Comment on qualitative differences and on whether higher reward always matches your own judgment. Discuss possible reward-model limitations when the reward signal and human judgment disagree.

Files to submit `q3_reward_model.py`, `q3_dpo.py`, `q3_bon.py`, and your PDF report.